Deliverable 3: Architecture & Design

System design, key decisions, what breaks at Nucor scale

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

Architecture & Design

The problem, sharpened

The prompt as given is broad: "help a shift supervisor prevent, anticipate, or react faster to unplanned downtime." I didn't want to build a generic "downtime dashboard." That's the kind of thing that sounds impressive in a slide and does nothing for anyone on a real shift. So I narrowed it hard, on purpose:

- **User:** a shift supervisor on a hot mill line, specifically. Not a reliability engineer doing offline analysis, not a plant manager looking at monthly KPIs. Someone watching live operation and deciding, in the moment, whether to intervene.
- **Failure mode:** roll stand bearing degradation. One mechanism, not "all possible downtime causes." Bearing wear is one of the most common and best-understood predictive-maintenance failure modes, which meant I could build something with a real, defensible precursor signal instead of hand-waving a dataset that just says "downtime: yes/no."
- **The decision it speeds up:** not "predict downtime" in the abstract, specifically "should I flag this stand for inspection before it fails on my shift." That's a concrete action a supervisor can actually take.

What this does NOT do

- It doesn't predict *all* downtime causes, just bearing degradation on a roll stand. A different failure mode (electrical fault, upstream material issue, human error) would need a different model and probably different signals entirely.
- It doesn't do automated shutdown or control-system intervention. It flags and explains; a human decides. I don't think an AI system should ever be the one pulling a mill offline on its own, and I say more about why in the security and risk doc.
- It doesn't handle real-time streaming at production scale. Update from the original submission: this used to say the model was "re-scored in batch, not a live sensor feed." That's no longer fully true. `app/live_feed.py` now ticks once a real minute, generates and scores a new row per stand, and the dashboard's Live Monitor tab reflects it without a page reload. Full story in the addendum near the bottom of this doc. It's still nowhere near production streaming though, one in-process background task and an in-memory buffer, not a real message bus with multiple consumers and sub-second latency, so the underlying gap is smaller than it was, not closed.
- It doesn't cover multi-mill, multi-vendor sensor normalization. Everything here assumes one consistent sensor schema across all stands, which is not how a real multi-division steelmaker's instrumentation actually looks.

Why Challenge 1 over the other two

I picked mill reliability over commercial/order-to-mill and internal-knowledge/onboarding because it's the one structurally closest to what the actual role does. The JD calls out ML-driven test data generation from real manufacturing process data and AI/ML anomaly detection for data quality. Predicting downtime from sensor drift is the same underlying shape of problem: time-series signals, anomaly detection, deciding what's a real alert versus noise. The knowledge/onboarding prompt is a RAG chatbot, which I think most candidates default to because it's the path of least resistance. I wanted to show the actual shape of this job instead.

System design

In production, the sensor-data box is a plant historian (OSIsoft PI is the common one in heavy industry, though Nucor's actual stack may differ), the feature pipeline runs continuously rather than in a batch script, and the alert layer would push into whatever a supervisor already watches: a control-room screen, not a separate app they have to remember to check.

There's also a testing and CI layer that isn't in the diagram above because it sits around the system, not inside it. `tests/dashboard.spec.js` is a Playwright suite covering the frontend's core flows (loads, shows real detection numbers, renders per-stand charts with actual content, the copilot button produces a grounded response), wired into GitHub Actions (`.github/workflows/ci.yml`), the accessible stand-in for what would be an Azure DevOps pipeline at

Nucor, gating merges the same way. It's caught real bugs during this build, and it also missed one that only showed up when the app was actually used on a real desktop machine: every chart was being fed roughly 13,000 undownsampled points, which rendered fine in Playwright's clean test browser but produced blank charts and multi-second lag on an actual desktop with a normal browser load. Automated tests prove a component responds; they don't fully substitute for someone actually using the thing. Full story, including what did and didn't get caught by automation, in `docs/ai-partnership-log.md`.

What I considered and rejected

- **A supervised deep model (LSTM/transformer over the raw time series) instead of an unsupervised anomaly detector.** Rejected for now. In real predictive maintenance you almost never have enough labeled failure events to train something that data hungry, and even in my synthetic set I only had 60 failures total. An unsupervised detector only needs to learn "normal," which I have plenty of. If this were headed to production with a year of real historian data and real failure logs, revisiting this would be a reasonable next step.
- **Picking IsolationForest by reasoning alone, without testing it against alternatives.** This is a real gap in the original submission. The reasoning above (why IsolationForest over a supervised model) was sound but I never actually compared IsolationForest against other unsupervised methods that fit the same "only needs normal data" constraint. `notebooks/model_comparison.ipynb` closes that gap: it trains a naive max abs z score baseline, IsolationForest, LocalOutlierFactor, OneClassSVM, and EllipticEnvelope on identical features and grades them on identical lead time and false alarm metrics. LocalOutlierFactor won outright: zero false alarms on the held out test window versus about 0.8% for IsolationForest, and roughly ten times more warning time before failure (about 9 hours median versus under an hour, later improved further, see below). The likely reason: IsolationForest partitions the whole feature space globally, so it tends to only flag a stand once a reading is extreme relative to everything. A degrading bearing's early trajectory is unusual relative to that specific stand's own recent neighborhood well before it becomes globally extreme, and LocalOutlierFactor's local density comparison catches that earlier. `app/detector.py` now trains LocalOutlierFactor in production as a result. This is still evaluated on synthetic data with one engineered failure signature, so it would need to be re run against real historian data before trusting the winner in production, and the caveats in the notebook (kernel and covariance methods only tested on an 8,000 row subsample, LocalOutlierFactor's higher scoring cost) are worth reading before treating this as settled.
- **Local density's blind spot on sustained failures, fixed with a backstop plus a labeling bug fix.** The same local-density property that makes LocalOutlierFactor catch subtle onset early also means it can stop flagging a failure hours in, once the fault's own recent minutes become each other's local "normal" (confirmed on real output: STAND-01 still showed 8-10.6 mm/s vibration six hours after failing, but its anomaly score had dropped back under threshold). Added an independent backstop to `app/detector.py`: force an alert if a raw signal sits more than 3.5 standard deviations from the stable, whole-training-period baseline for 20 consecutive minutes, regardless of what the model says. First test run pushed the false alarm rate from 0.00% to 4.18%, which led to finding a real, pre-existing bug in `app/labels.py`: "normal" was only ever defined by distance to the NEXT failure, so rows still recovering from a PREVIOUS failure (the generator holds a stand at failed values for the rest of the day once it fails) were being counted as clean baseline. Fixed by adding a backward-looking `time_since_failure_min` and requiring distance from a failure in both directions. Result: false alarm rate down to 0.05%, and median lead time actually improved from 562 to 687 minutes, because the training data itself got cleaner as a side effect of the fix. Full story in `docs/ai-partnership-log.md`'s second addendum. Not re-verified: whether this labeling fix would shift the bake off's model ranking, since the notebook wasn't re run against the corrected data.
- **Real-time streaming (Kafka or similar) instead of batch scoring.** Rejected for the prototype. It's the right architecture eventually, but building streaming infrastructure to process a synthetic CSV would have been effort

spent on plumbing instead of on the actual detection problem. Documented as a known gap, not silently ignored.

- **A single global anomaly threshold vs. per-stand thresholds.** I used one global threshold across all 6 stands. That's a real simplification; different stands likely have different normal baselines and different sensitivities in a real mill. Flagged as a scale limitation below, not solved here.
- **Frequency-domain features (FFT on vibration) instead of just rolling stats.** Vibration analysis in real predictive maintenance often goes into frequency space to catch specific fault signatures. I started simpler on purpose (rolling mean, std, rate-of-change) to get an honest end-to-end baseline working before reaching for something fancier. It worked well enough (11/11 detection on held-out data) that I didn't feel the need to escalate for this prototype.
- **Streamlit, then React and FastAPI, for the interface.** I built the first working UI in Streamlit because it let me validate whether the ML approach worked at all in an afternoon instead of a week. Once the detector was proven out (11/11 detection, a real evaluated tradeoff curve), I rebuilt the interface properly: a FastAPI backend serving the scored data and proxying the copilot call, and a React and TypeScript frontend for the actual supervisor-facing surface. Prototyping fast in Streamlit and then investing in a real frontend once the underlying idea is validated is a deliberate two-stage decision, not scope creep.

Where this breaks at Nucor scale

- **Per-stand and per-mill calibration.** A single trained model and single threshold across all stands is a prototype-scale shortcut. Real stands have different equipment age, load profiles, and sensor calibration; this would need per-stand (or at least per-mill-type) baselines, not one global model.
- **Sensor vendor heterogeneity.** I assumed one clean, consistent 5-signal schema. A real multi-division steelmaker almost certainly has different sensor vendors, tag naming conventions, and sampling rates across sites. Normalizing that is a real data-engineering project on its own, not a footnote.
- **Model retraining and drift.** "Normal" operation drifts over time: new equipment, seasonal effects, process changes. A production version needs a retraining cadence and drift monitoring, or the model quietly goes stale and either misses real failures or false-alarms more and more.
- **Alert fatigue across shifts.** A 1% false-alarm rate on one stand's test window sounds fine. Multiply that across every stand, every mill, every shift, continuously, and it adds up fast. This is the exact kind of thing that gets a tool ignored within a month if it isn't actively managed. I don't have a full solution to this in the prototype; I flag it as the single biggest adoption risk in the security and risk doc.
- **Integration with existing workflows.** This assumes a supervisor checks a dashboard. In practice it would need to land inside whatever they already use, likely paging into an existing alarm/SCADA system rather than being one more screen to remember.

Addendum: from a static demo to a live one

The original submission was a single static CSV, already scored once and never touched again. Fine for proving the detection idea works, bad for actually showing what "help a supervisor watch a live line" feels like, because nothing ever changed while you looked at it. Added three new pieces on top of the same trained model to fix that, no retraining involved anywhere in this addendum.

`app/live_feed.py`, a background feed that actually moves. Ticks once every real minute, generates one new row per stand using the same baseline and failure ramp math as the training generator, scores it with the real trained detector, and appends it to `data/synthetic/live_readings.csv`. Failures still happen at random, but I deliberately detuned the odds and speed from the training generator's literal numbers. At the real 22 percent per stand per day rate I trained on, a demo could sit at "all normal" for hours before anything happened at all, which I found out by actually

leaving it running and watching nothing happen. Recalibrated to about 1 percent per stand per minute and a 10 to 40 minute degrade window instead of 6 to 48 hours, and said so plainly in the code as tuned for watchability, not statistical realism, since conflating the two would be dishonest about what the number means.

`app/historical.py` **and a rolling data window instead of a fixed date.** The live feed only knows about "now" onward, so a viewer picking Today, Past week, Last month, or a custom range on the dashboard needed the historical training data to actually reach back from whenever "now" happens to be, not sit frozen at whatever calendar date I first ran the generator on. `data/generate_synthetic_data.py` now anchors its 45 day window to end yesterday, computed off the real clock at generation time. And since closing the app for a few days would otherwise leave a visible gap between wherever the old data stopped and today, `historical.ensure_fresh()` checks that gap on every API startup and silently regenerates if it's more than a day stale, so reopening this after a weekend heals itself instead of showing a hole.

I almost did something wasteful here. My first instinct after changing the generator's dates was to also retrain, since "new data" sounded like it needed a new model. It doesn't, and I only realized that because the user asked why I was about to. The generator has a fixed random seed, so regenerating with different calendar labels produces byte-identical signal sequences, only the timestamps move. The trained model never sees a calendar date, only engineered rolling stats, so it's exactly as valid against freshly dated data as it was against the original, and retraining would have been pure waste that also risked quietly shifting the documented 11/11 detection and 0.05 percent false alarm numbers for no actual reason. Skipped it. Separately, scoring the full 45 day set even once still took about 100 seconds through LocalOutlierFactor, far too slow to do inside a request, so that result is cached to `data/synthetic/full_scored.csv` after the first computation instead of recomputed on every server start.

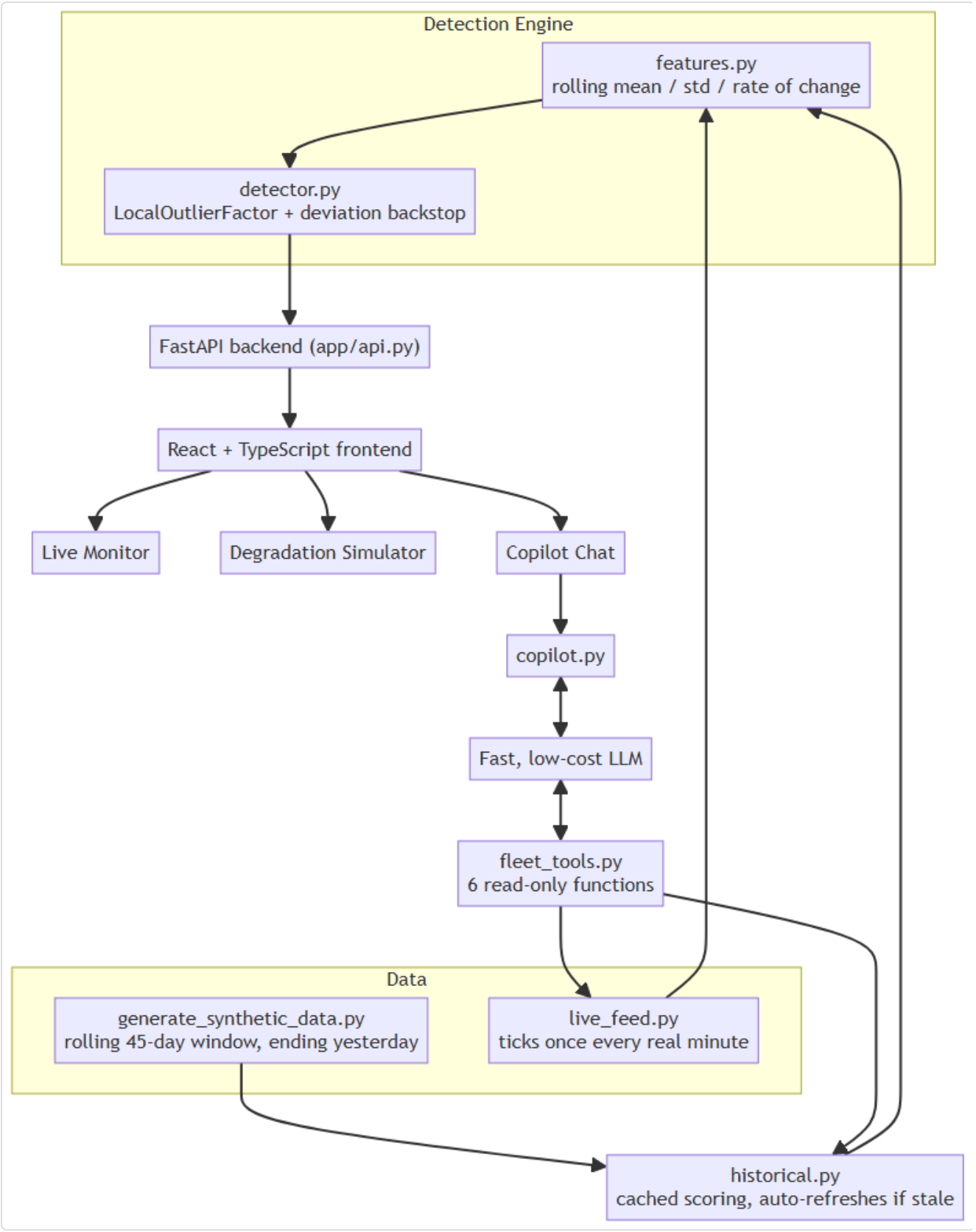
`app/simulator.py` **, a way to see the model react on demand.** Waiting on the live feed's own randomness is fine for ambient monitoring but useless for actually demonstrating detection behavior on request. The simulator is one virtual stand with five editable signal fields that auto correlate along the same normal to failure curve (`app/signal_profile.py`) the real generator uses, plus a timed degradation run that ramps toward failure over however many minutes you choose, scored live by the real detector at every simulated minute. Tested it end to end rather than assuming: a 10 minute run alerted 1 minute before the end, a 30 minute run alerted 15 minutes before the end. That's an honest, reproducible demonstration that slower degradation buys proportionally more warning time, which is the entire point of this project, not something I had to fake.

A fleet wide "ask the copilot," and why it isn't vector search RAG. The original copilot only ever answered one fixed question about one stand. Extending it to free text questions across the whole fleet (compare two stands, which one's alerting, what's causing it, how often) looked at first like a RAG problem, but it isn't one at this scale. With only 6 stands, the entire fleet's current state trivially fits in a single prompt, so embeddings and similarity search solve a problem (a corpus too big to fit in context) that doesn't exist here. Instead `app/fleet_tools.py` exposes six narrow, read only Python functions, current reading, all current readings, last alert, alert frequency, alert cause, and a trend check, as Claude tool use functions. The model decides which ones a given question actually needs, calls them itself, and only writes an answer from what came back. Guardrail details and the real bugs this surfaced are in `docs/ai-partnership-log.md` and `docs/security-risk.md` .

The copilot got its own tab, with actual conversation memory, and the sidebar got something more honestly useful in its place. The copilot started life as a small widget stuffed into the stand detail sidebar: one button, one free text box, one visible answer at a time. That was fine for a quick lookup, wrong for anything that reads as an actual assistant. It now lives on its own tab (`web/src/components/FleetCopilotPage.tsx`) with real chat bubbles and, unlike the sidebar version, genuine multi turn memory: the whole visible conversation is sent back to Claude on every question, `app/copilot.py` 's `answer_fleet_question` now takes the full message history instead of a single string, which is what lets a follow up like "and what about its bearing temp" resolve "its" correctly instead of treating every question as the first one ever asked. In its old spot in the sidebar, `web/src/components/CurrentReadingsPanel.tsx`

now shows the selected stand's live sensor values directly, numbers a supervisor glancing at the sidebar actually wants, not a chat entry point that's now one tab away instead of a click away.

What it looks like, running


Detection Engine
features.py
rolling mean / std / rate of change
detector.py
LocalOutlierFactor + deviation backstop
FastAPI backend (app/api.py)
React + TypeScript frontend
Live Monitor
Degradation Simulator
Copilot Chat
copilot.py
Fast, low-cost LLM
fleet_tools.py
6 read-only functions
Data
generate_synthetic_data.py
rolling 45-day window, ending yesterday
live_feed.py
ticks once every real minute
historical.py
cached scoring, auto-refreshes if stale

Mills-Operation / Reliability Console

Prototype for a hot mill shift supervisor. Flags roll stand bearing degradation before it becomes unplanned downtime.

FLEET STATUS

Click a stand to see its live signal charts.

STAND-01 NORMAL

VIBRATION 3.6 mm/s RMS	BEARING TEMP 60.8 °C
MOTOR CURRENT 405.6 A	LINE SPEED 622.8 m/min
COOLANT PRESSURE 80.6 psi	ANOMALY SCORE 1.37

STAND-02 NORMAL

VIBRATION 2.2 mm/s RMS	BEARING TEMP 57.5 °C
MOTOR CURRENT 384.9 A	LINE SPEED 616.5 m/min
COOLANT PRESSURE 84.5 psi	ANOMALY SCORE 1.08

STAND-03 NORMAL

VIBRATION 3.0 mm/s RMS	BEARING TEMP 56.6 °C
MOTOR CURRENT 413.8 A	LINE SPEED 643.9 m/min
COOLANT PRESSURE 82.6 psi	ANOMALY SCORE 1.20

STAND-04 NORMAL

VIBRATION 1.7 mm/s RMS	BEARING TEMP 61.5 °C
MOTOR CURRENT 407.1 A	LINE SPEED 627.8 m/min
COOLANT PRESSURE 87.9 psi	ANOMALY SCORE 1.07

STAND-05 NORMAL

VIBRATION 2.2 mm/s RMS	BEARING TEMP 58.6 °C
MOTOR CURRENT 410.6 A	LINE SPEED 602.9 m/min
COOLANT PRESSURE 82.0 psi	ANOMALY SCORE 1.07

STAND-06 NORMAL

VIBRATION 2.2 mm/s RMS	BEARING TEMP 58.3 °C
MOTOR CURRENT 414.1 A	LINE SPEED 622.9 m/min
COOLANT PRESSURE 84.3 psi	ANOMALY SCORE 1.04

MODEL PERFORMANCE (HELD-OUT TEST)

Failures detected	11/11
False alarm rate	0.05%
Alert threshold	1.289

← Back to fleet

Mills-Operation / Reliability Console

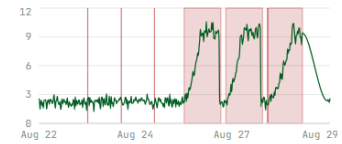
Live monitor | Degradation simulator | Copilot

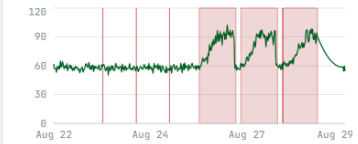
STAND-01

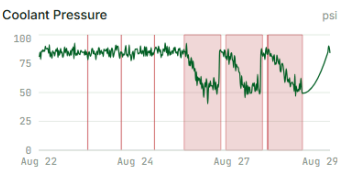
Today | Past week | Last month | Custom range

FLEET

- STAND-01 NORMAL
- STAND-02 NORMAL
- STAND-03 NORMAL
- STAND-04 NORMAL
- STAND-05 NORMAL
- STAND-06 NORMAL







STATUS

Latest anomaly score	1.367
Alert threshold	1.288

CURRENT READINGS

VIBRATION 3.59 mm/s RMS	BEARING TEMP 60.80 °C
MOTOR CURRENT 405.60 A	LINE SPEED 622.78 m/min
COOLANT PRESSURE 80.59 psi	ANOMALY SCORE 1.367

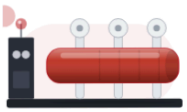
← Back to fleet

Mills-Operation / Reliability Console

Live monitor | Degradation simulator | Copilot

DEGRADATION SIMULATOR

One virtual stand. Change a signal by hand or run a timed degradation and watch the real trained model score it live, minute by minute.



ALERT

anomaly score 24.094 / threshold 1.288

minute 8 of 15

Alert triggered at minute 5 of 15

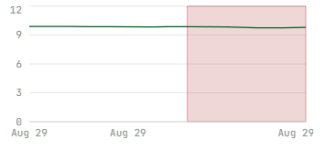
Degrade over 15 minutes

Degrading... | Reset

SIGNAL VALUES

Vibration (mm/s RMS)	9.9
Bearing Temp (°C)	93.863
Motor Current (A)	473.288
Line Speed (m/min)	556.712
Coolant Pressure (psi)	53.356

Vibration during this run — mm/s RMS



← Back to fleet

Mills-Operation / Reliability Console

Live monitor | Degradation simulator | Copilot

COPILOT

Ask about any stand: compare readings, check who's alerting, look up alert history. Answers are grounded in real fleet data, never invented.

Clear chat

Explain STAND-01's status

Which stand has the highest bearing temperature?

STAND-01 has the highest bearing temperature at **60.75°C**.

Ask about the fleet...

Send